PROJET PERSO · STM32 · C++ BARE METAL

Enes · octobre 2026

Télémétrie RS

Mesurer. Comprendre. Progresser.

Mon propre système d'acquisition de données pour une Mégane 3 RS Trophy : capteurs, firmware et analyse, tout est construit de zéro, registre par registre.

L'idée en 30 secondes

Sur circuit, ce qui fait gagner des dixièmes, ce n'est pas le ressenti : ce sont les données. Je construis un boîtier qui enregistre, pendant un tour, **où est la voiture**, **à quelle vitesse elle va** et **combien de G elle encaisse**. Je récupère ensuite le fichier sur PC pour rejouer le tour, courbe par courbe, et comparer deux passages. Le tout tourne sur un microcontrôleur STM32, avec un firmware que j'écris moi-même en **C++**.

01
Mesurer

Un GPS (visé : 10 Hz) pour le tracé, une centrale inertielle (IMU) pour les G latéraux, longitudinaux et verticaux, et plus tard le bus CAN de la voiture pour le régime et la vitesse moteur.

02
Comprendre

Chaque mesure est horodatée sur une base de temps commune et enregistrée sur carte SD. Python s'occupe du reste : tracé du circuit, courbes, comparaison de deux tours.

03
Construire

Aucune bibliothèque ne cache le matériel. Je programme les périphériques (GPIO, UART, SPI, I2C, timers) directement par leurs registres.

Pourquoi le faire soi-même ?

L'écran d'origine de la voiture (R.S. Monitor) affiche certaines données, mais ne les exporte pas en brut. Des solutions du commerce existent (AiM, RaceBox, VBOX…), mais elles livrent un produit fini. Ici, l'objectif est de **posséder toute la chaîne** : je choisis la fréquence d'échantillonnage, je vois chaque octet passer, et je peux expliquer chaque ligne de code. C'est aussi, à petite échelle, exactement le métier d'ingénieur télémétrie : fusionner plusieurs sources de mesure et en tirer un tour lisible.

Le matériel

STM32 Nucleo F4 Carte principale	**Module GPS** Déjà acquis	**IMU (type MPU-6050)** À acquérir	**Carte SD + support** À acquérir	**Interface CAN** Plus tard

Les objectifs

Techniques

- Acquérir position et vitesse par GPS (trames NMEA, 10 Hz visés)
- Mesurer les accélérations sur 3 axes avec une IMU, filtrées et calibrées
- Horodater toutes les mesures avec un timer matériel commun
- Enregistrer en continu sur carte SD au format CSV, sans corrompre le fichier
- Visualiser le tour sur PC : tracé, courbes, comparaison de deux tours
- Phase 2 : lire le bus CAN (OBD2) en lecture seule

Apprentissage

- Comprendre le microcontrôleur à bas niveau : mémoire, bus, horloges, interruptions
- Écrire les drivers de A à Z : GPIO, SPI, I2C, USART
- Maîtriser le C++ moderne en l'appliquant à du vrai matériel
- Savoir déboguer : débogueur, analyseur logique, traces série

Carrière

- Une pièce maîtresse de portfolio : une chaîne d'acquisition complète sur une vraie voiture
- Le métier d'ingénieur télémétrie et performance, en miniature
- Des compétences transférables à tout l'embarqué automobile

Partage

- Documenter sur GitHub avec un README clair, des schémas et les jalons
- Envisager d'ouvrir le code en open source
- Rester honnête sur l'état d'avancement : un jalon à la fois

Pourquoi du C++ plutôt que du C ?

Le firmware est écrit en **C++ moderne, directement sur le matériel, sans bibliothèque HAL**. Je m'appuie sur un cours de référence en C, que je réécris en classes C++, driver par driver. Résultat : le même niveau de contrôle, avec un code plus sûr et plus lisible.

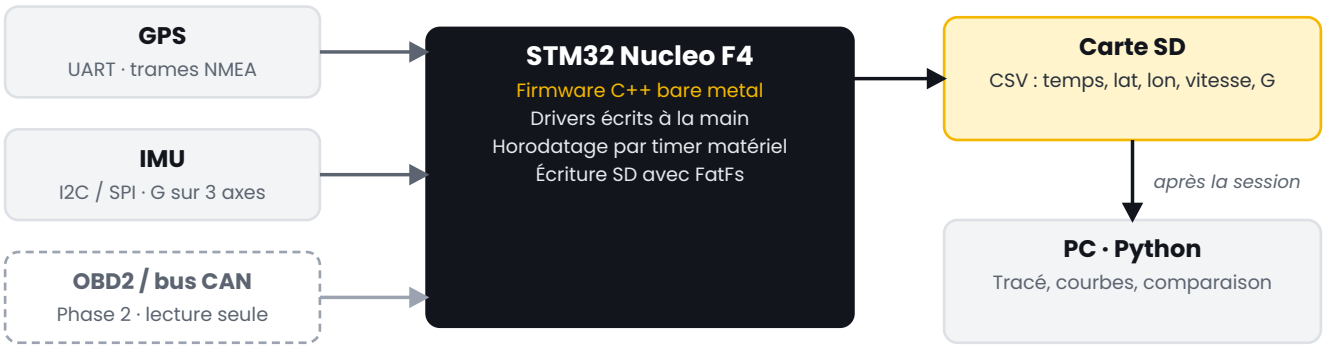
Des drivers propres
Une classe par périphérique (Gpio, Spi, I2c, Uart). La configuration passe par le constructeur, les ressources sont libérées automatiquement (RAII).

Des erreurs vues à la compilation
Les enum class et les types forts empêchent de mélanger deux modes ou deux registres sans que le compilateur ne proteste.

Des calculs faits avant l'exécution
Avec constexpr, les prescalers et les baud rates sont calculés à la compilation, sans aucun coût sur la carte.

Des abstractions légères
Des templates, par exemple pour un buffer circulaire générique, et de la composition plutôt que de l'héritage lourd.

De la mesure à l'analyse



La feuille de route

Étape	Ce qu'on construit	Livrable	Cible
Jalon 0 Fondations	Les bases du microcontrôleur (débogage, mémoire, bus, horloges, interruptions), puis les classes Rcc et Gpio, avec un bouton en interruption.	Classes Rcc et Gpio testées sur la carte	mi-novembre
Jalon 1 GPS	USART en classe C++, puis décodage des trames NMEA : position et vitesse en direct, viser 10 Hz.	La position s'affiche en direct sur le PC	fin novembre
Jalon 2 Carte SD	SPI en classe C++, puis écriture sur carte SD avec FatFs : trames GPS en CSV, fermeture propre du fichier. C'est le jalon le plus technique.	Un fichier CSV complet après une session	mi-janvier
Jalon 3 IMU	I2C en classe C++, puis lecture de l'accéléromètre, filtrage du bruit et calibration au repos.	G latéraux et longitudinaux en direct	fin février
Jalon 4 Visualisation	Horodatage commun, CSV GPS + IMU, puis Python et pandas : tracé du circuit, courbes de vitesse et de G, deux tours superposés.	L'image d'un tour, avec ses phases d'accélération et de freinage	mars-avril
Jalon 5 Test réel	Montage fixé solidement, alimentation stable, premier roulage prudent.	Un premier vrai tour enregistré dans la Mégane	été 2027
Phase 2 CAN / OBD2	Lecture seule du bus CAN pour le régime et la vitesse moteur, puis filtre de Kalman et visualisation avancée.	Données moteur synchronisées avec GPS et IMU	après

Calendrier indicatif : il tient compte d'une pause de Noël et des partiels de fin janvier, et peut glisser de quelques semaines.

Ce que ça donnera

Le tracé

La forme du circuit reconstruite depuis le GPS, colorée selon la vitesse ou le freinage.

Les courbes

Vitesse, G latéral et G longitudinal, en fonction du temps ou de la distance.

Les comparaisons

Deux tours superposés pour voir où se gagnent, et où se perdent, les dixièmes.

Et après ?

Le cœur du projet reste d'apprendre et de construire. À terme, je veux le documenter proprement sur GitHub et envisager de l'ouvrir en open source. Une application dédiée, voire un produit, c'est une possibilité lointaine, pas un objectif d'aujourd'hui : le marché est déjà occupé et je préfère d'abord faire quelque chose de solide, dont je comprends chaque ligne.

Ça t'intéresse ? Un avis, une idée de capteur ou de visualisation, un coup de main pour tester sur circuit : dis-moi ce que tu en penses.